

# Introduction to Decision-Focused Learning for airlines

## Part 2: Decision-Focused Learning

Léo Baty

22/09/2026



ÉCOLE NATIONALE DES  
**PONTS**  
ET CHAUSSÉES



IP PARIS

**AIRFRANCE** 

## Part 2: outline

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 DFL for stochastic tail assignment
- 4 The JuliaDecisionFocusedLearning ecosystem
- 5 **Demo:** contextual stochastic knapsack problem
- 6 Practice session 2

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 DFL for stochastic tail assignment
- 4 The JuliaDecisionFocusedLearning ecosystem
- 5 **Demo:** contextual stochastic knapsack problem
- 6 Practice session 2

# Decision-Focused Learning policies

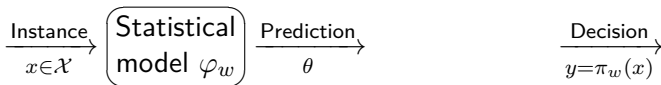
- **Learning problem:** find  $\pi$  such that  $\pi(x)$  is “good”.

Instance  
 $\xrightarrow{x \in \mathcal{X}}$

Decision  
 $\xrightarrow{y = \pi(x)}$

# Decision-Focused Learning policies

- **Learning problem:** find  $w$  such that  $\pi_w(x)$  is “good”.

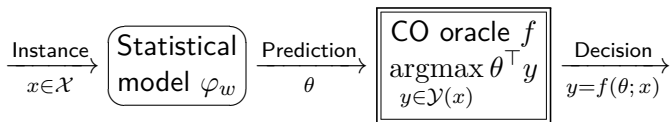


What is needed to build a DFL policy?

1. A **statistical model**  $\varphi_w$  (GLM, NN, GNN, ...).

# Decision-Focused Learning policies

- **Learning problem:** find  $w$  such that  $\pi_w(x) = f(\varphi_w(x); x)$  is “good”.

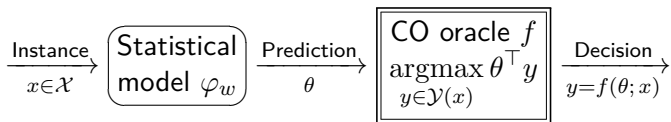


What is needed to build a DFL policy?

1. A **statistical model**  $\varphi_w$  (GLM, NN, GNN, ...).
2. A **CO oracle**  $f$ .

## Decision-Focused Learning policies

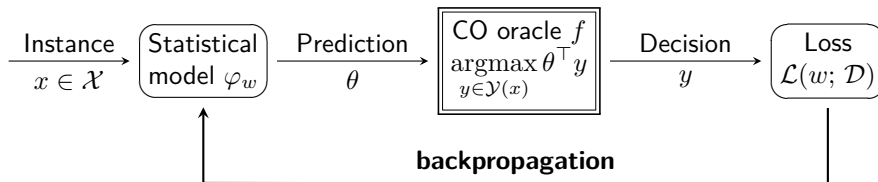
- **Learning problem:** find  $w$  such that  $\pi_w(x) = f(\varphi_w(x); x)$  is “good”.



What is needed to build a DFL policy?

1. A **statistical model**  $\varphi_w$  (GLM, NN, GNN, ...).
2. A **CO oracle**  $f$ .
3. A **dataset**  $\mathcal{D}$ .
4. A **learning algorithm**.

# Training the policy



## Supervised learning

Dataset:  $\mathcal{D} = (x_i, \bar{y}_i)_{i \in [N]}$

Learning problem:

$$\min_w \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\varphi_w(x_i)), \bar{y}_i)$$

## Empirical Risk Minimization

Dataset:  $\mathcal{D} = (x_i)_{i \in [N]}$

Learning problem:

$$\min_w \frac{1}{N} \sum_{i=1}^N c(f(\varphi_w(x_i)))$$

## Why it is hard

The CO oracle  $f: \theta \mapsto \operatorname{argmax}_{y \in \mathcal{Y}(x)} \theta^\top y$  is **piecewise constant** in  $\theta$ :

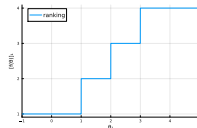
- tiny changes in  $\theta \rightarrow$  same decision  $\rightarrow$  gradient = 0,
- at the boundaries  $\rightarrow$  the decision jumps  $\rightarrow$  gradient undefined.

No usable gradient  $\implies$  no end-to-end training.

$$\text{ranking: } \theta \mapsto \operatorname{argmax}_{y \in \sigma(n)} \theta^\top y,$$

$\sigma(n)$ : permutations of  $\{1, \dots, n\}$  (ranking vectors)

$$\theta_1 \mapsto [\text{ranking}([\theta_1, 3, 1, 2])]_1$$



This is the whole problem DFL solves: make the optimization layer **differentiable** (perturbation, regularization, ...).

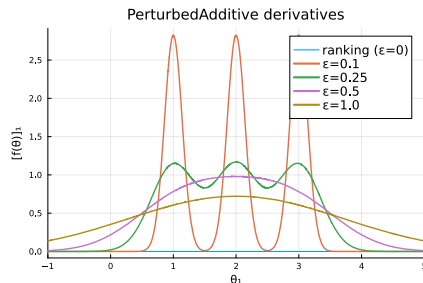
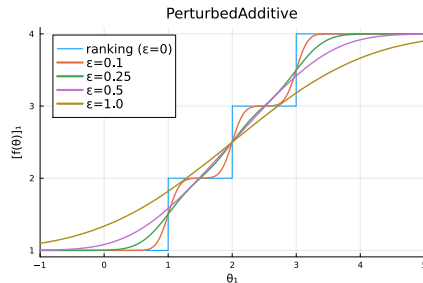
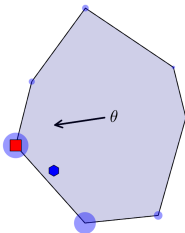
- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 DFL for stochastic tail assignment
- 4 The JuliaDecisionFocusedLearning ecosystem
- 5 **Demo:** contextual stochastic knapsack problem
- 6 Practice session 2

# Smoothing by perturbation

$$f(\theta) = \operatorname{argmax}_{y \in \mathcal{Y}(x)} \theta^\top y$$

$$\hat{f}(\theta) = \mathbb{E}_Z \left[ \operatorname{argmax}_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right]$$

$$\varepsilon \in \mathbb{R}_+, Z \sim \mathcal{N}(0, I)$$



## Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where  $\Omega$  is a convex regularization function.

The perturbed  $\hat{f}$  from the previous slide is a regularized argmax for this  $\Omega$ : this is what makes the Fenchel-Young machinery apply.

## Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where  $\Omega$  is a convex regularization function.

Perturbation case:

$$\Omega^*(\theta) = \mathbb{E}_Z \left[ \max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right].$$

The perturbed  $\hat{f}$  from the previous slide is a regularized argmax for this  $\Omega$ : this is what makes the Fenchel-Young machinery apply.

# Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where  $\Omega$  is a convex regularization function.

Perturbation case:

$$\Omega^*(\theta) = \mathbb{E}_Z \left[ \max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right].$$

The perturbed  $\hat{f}$  from the previous slide is a regularized argmax for this  $\Omega$ : this is what makes the Fenchel-Young machinery apply.

Perturbed case notation:

$$\mathcal{L}_\Omega = \mathcal{L}_\varepsilon^{FYL}.$$

Non-optimality of  $\bar{y}$   
as a solution of the  
regularized prediction problem

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \max_{y \in \operatorname{conv}(\mathcal{Y}(x))} (\theta^\top y - \Omega(y)) - (\theta^\top \bar{y} - \Omega(\bar{y}))$$

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \Omega^*(\theta) + \Omega(\bar{y}) - \theta^\top \bar{y}$$

# Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where  $\Omega$  is a convex regularization function.

Perturbation case:

$$\Omega^*(\theta) = \mathbb{E}_Z \left[ \max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right].$$

Non-optimality of  $\bar{y}$   
as a solution of the  
regularized prediction problem

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \max_{y \in \operatorname{conv}(\mathcal{Y}(x))} (\theta^\top y - \Omega(y)) - (\theta^\top \bar{y} - \Omega(\bar{y}))$$

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \Omega^*(\theta) + \Omega(\bar{y}) - \theta^\top \bar{y}$$

The perturbed  $\hat{f}$  from the previous slide is a regularized argmax for this  $\Omega$ : this is what makes the Fenchel-Young machinery apply.

Perturbed case notation:

$$\mathcal{L}_\Omega = \mathcal{L}_\varepsilon^{FYL}.$$

Properties that make SGD tractable

- $\mathcal{L}_\Omega(\theta; \bar{y}) \geq 0$
- $\mathcal{L}_\Omega(\theta; \bar{y}) = 0 \Leftrightarrow \bar{y} = \nabla \Omega^*(\theta)$
- Convex in  $\theta$
- $\nabla_\theta \mathcal{L}_\Omega(\theta; \bar{y}) = \hat{f}_\Omega(\theta) - \bar{y}$

## Building a differentiable loss

Generalization for  $f: \theta \mapsto \underset{y}{\operatorname{argmax}} \theta^\top g(y) + h(y)$ . Setting  $g(y) = y$  and  $h = 0$  recovers the Fenchel-Young loss  $\mathcal{L}_\Omega$  from the previous slide, with  $\Omega$  induced by the perturbation.

Theorem [Berthet et al. 2020, generalization to non-linear costs by Baty, Jungel, Klein, Parmentier, Schiffer, Transportation Science 2024]

The perturbed loss function

$$\mathcal{L}_\varepsilon(\theta, \bar{y}) = \mathbb{E} \left[ \max_{y \in \mathcal{Y}} (\theta + \varepsilon Z)^\top g(y) + h(y) \right] - (\theta^\top g(\bar{y}) + h(\bar{y}))$$

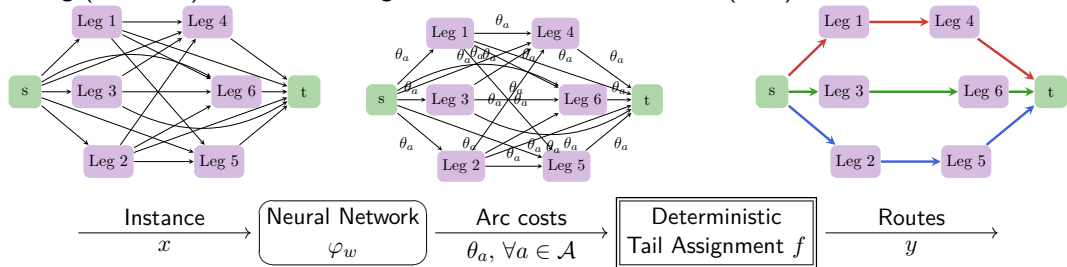
with  $\varepsilon \in \mathbb{R}_+$ , and  $Z \sim \mathcal{N}(0, I_d)$ , is convex and differentiable in  $\theta$

$$\begin{aligned} \nabla_\theta \mathcal{L}_\varepsilon(\theta, \bar{y}) &= \mathbb{E} \left[ g \left( \underset{y \in \mathcal{Y}}{\operatorname{argmax}} (\theta + \varepsilon Z)^\top g(y) + h(y) \right) \right] - g(\bar{y}) \\ &= \mathbb{E}[g(f(\theta + \varepsilon Z))] - g(\bar{y}) \end{aligned}$$

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 **DFL for stochastic tail assignment**
- 4 The JuliaDecisionFocusedLearning ecosystem
- 5 **Demo:** contextual stochastic knapsack problem
- 6 Practice session 2

## Target 1: imitating integer solutions

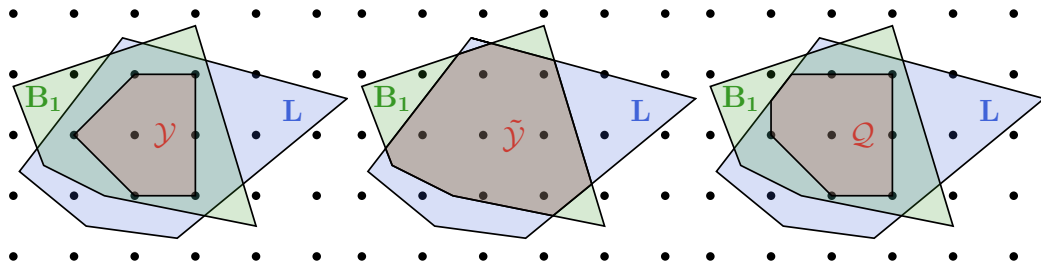
We learn the arc costs of a fast deterministic tail assignment solver so that its routes reproduce those of the slow stochastic solver. Two ways to build the training targets: integer solutions from diving (this slide), or the column generation relaxation solutions (next).



Fenchel-Young loss over integer solutions ( $\bar{y} \in \mathcal{Y}(x)$ ):

$$\mathcal{L}_y^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[ \max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y},$$

## Target 2: imitating column generation relaxation solutions



- $Y$ : feasible (integer) routes
- $\tilde{Y}$ : relaxed integer constraints
- $Q$ : column generation relaxation

For the stochastic tail assignment:  $Q = \tilde{Y}$  (flow constraints). The convex hull of source-to-sink paths in a directed acyclic graph is exactly the unit flow polytope, so the Dantzig-Wolfe reformulation does not strengthen the relaxation here.

## Target 2: imitating column generation relaxation solutions

Fenchel-Young loss over column generation relaxation solution ( $\bar{y} \in \mathcal{Q} = \tilde{\mathcal{Y}}$ ):

$$\mathcal{L}_{\tilde{\mathcal{Y}}}^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[ \max_{y \in \mathcal{Q}} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y} = \mathbb{E}_Z \left[ \max_{y \in \tilde{\mathcal{Y}}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y}.$$

## Target 2: imitating column generation relaxation solutions

Fenchel-Young loss over column generation relaxation solution ( $\bar{y} \in \mathcal{Q} = \tilde{\mathcal{Y}}$ ):

$$\mathcal{L}_{\tilde{\mathcal{Y}}}^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[ \max_{y \in \mathcal{Q}} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y} = \mathbb{E}_Z \left[ \max_{y \in \tilde{\mathcal{Y}}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y}.$$

During training, we only need to use an LP **relaxation**:

$$\tilde{f}: \theta \longmapsto \operatorname{argmax}_{y \in \tilde{\mathcal{Y}}(x)} \theta^\top y$$

## Target 2: imitating column generation relaxation solutions

Fenchel-Young loss over column generation relaxation solution ( $\bar{y} \in \mathcal{Q} = \tilde{\mathcal{Y}}$ ):

$$\mathcal{L}_{\tilde{\mathcal{Y}}}^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[ \max_{y \in \mathcal{Q}} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y} = \mathbb{E}_Z \left[ \max_{y \in \tilde{\mathcal{Y}}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y}.$$

During training, we only need to use an LP **relaxation**:

$$\tilde{f}: \theta \longmapsto \operatorname{argmax}_{y \in \tilde{\mathcal{Y}}(x)} \theta^\top y$$

**Upsides:**

- **faster** training
- no need to find good integer solutions, **column generation relaxation is enough**

# Experiment setup

## Instances:

- Train/validation: small instances
- Test: larger instances

**Features:** distributional information about connection slacks

## Solutions:

- $\mathcal{D}^{\tilde{\mathcal{Y}}}$ : dataset with solutions of the column generation relaxation with 100 scenarios
- $\mathcal{D}^{\mathcal{Y}}$ : dataset with integer solutions (from diving heuristic) with 100 scenarios

## Results (smaller is better)

| <b>Evaluation dataset</b> | <b>Train</b> | <b>Val</b> | <b>Test</b> | <b>Diving heuristic</b> |
|---------------------------|--------------|------------|-------------|-------------------------|
| <b>Average run time</b>   | 31s          | 31s        | 46s         | 2061s                   |

Diving heuristic: this morning's baseline, 100 scenarios (colgen.tex).

Orders of magnitude faster (46s vs. 2061s) at about 2% quality loss compared to the diving heuristic.

## Results (smaller is better)

| <b>Evaluation dataset</b> | Train | Val | Test | <b>Diving heuristic</b> |
|---------------------------|-------|-----|------|-------------------------|
| <b>Average run time</b>   | 31s   | 31s | 46s  | 2061s                   |

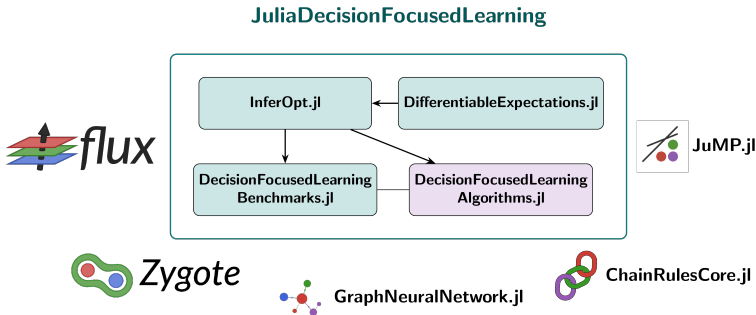
Diving heuristic: this morning's baseline, 100 scenarios (colgen.tex).

| <b>Training dataset</b>                  | $\mathcal{D}^{\tilde{\mathcal{Y}}}$ |      |      | $\mathcal{D}^{\mathcal{Y}}$ |      |             |
|------------------------------------------|-------------------------------------|------|------|-----------------------------|------|-------------|
| <b>Evaluation dataset</b>                | Train                               | Val  | Test | Train                       | Val  | Test        |
| <b>Average gap (to diving heuristic)</b> | 1.7%                                | 2.2% | 2.9% | 1.7%                        | 1.5% | <b>2.3%</b> |

Orders of magnitude faster (46s vs. 2061s) at about 2% quality loss compared to the diving heuristic.

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 DFL for stochastic tail assignment
- 4 The JuliaDecisionFocusedLearning ecosystem
- 5 Demo: contextual stochastic knapsack problem
- 6 Practice session 2

## Three main packages



- |                               |                                                          |
|-------------------------------|----------------------------------------------------------|
| <code>InferOpt.jl</code>      | differentiable CO layers + losses (core building blocks) |
| <code>DFLBenchmarks.jl</code> | standardized problems behind one interface               |
| <code>DFLAlgorithms.jl</code> | black-box training algorithms in one line                |

# A tour of DFLBenchmarks

Three main categories of benchmarks:

| Family                             | Benchmarks                                                                                         |
|------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>AbstractBenchmark</b>           | Argmax, Argmax2D, Ranking, SubsetSelection, PortfolioOptimization, FixedSizeShortestPath, Warcraft |
| <b>AbstractStochasticBenchmark</b> | StochasticVehicleScheduling, ContextualStochasticArgmax                                            |
| <b>AbstractDynamicBenchmark</b>    | DynamicVehicleScheduling, DynamicAssortment, Maintenance                                           |

# A tour of DFLBenchmarks

Three main categories of benchmarks:

| Family                             | Benchmarks                                                                                         |
|------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>AbstractBenchmark</b>           | Argmax, Argmax2D, Ranking, SubsetSelection, PortfolioOptimization, FixedSizeShortestPath, Warcraft |
| <b>AbstractStochasticBenchmark</b> | StochasticVehicleScheduling, ContextualStochasticArgmax                                            |
| <b>AbstractDynamicBenchmark</b>    | DynamicVehicleScheduling, DynamicAssortment, Maintenance                                           |

- **Evaluate a new algorithm:** pick a benchmark, train a policy, evaluate
  - ▶ change one line to change the problem
- **Implement a new problem** using the common interface.
  - ▶ gives access to the whole ecosystem features

The interface is designed to be easy to extend.

## Using a benchmark

A benchmark hands you the needed **three components** to initialize a DFL policy:

```
bench = StochasticVehicleSchedulingBenchmark() # Choose a benchmark problem
```

```
# 1. Statistical model: a Flux model (features -> values)
```

```
model = generate_statistical_model(bench)
```

```
# 2. CO oracle: the combinatorial maximizer
```

```
maximizer = generate_maximizer(bench)
```

```
# 3. data: a dataset of (features, target) samples
```

```
train_data = generate_dataset(bench, 100; seed=1)
```

```
# 4. apply your learning algorithm here
```

```
# ...
```

Swap `bench` to a new problem, **same three lines**.

## A dataset is a vector of DataSample

Each DataSample bundles everything about one instance:

```
struct DataSample
    x          # input features
    y          # target solution (if it applies)
    context    # observed input data (instance, context information, ...)
    extra      # any extra non-solver data: scenario, reward, ...
end
```

Applying the policy to one sample:

```
sample = train_data[1]
 $\theta$  = model(sample.x)          # features    -> prediction
y = maximizer( $\theta$ ; sample.context...) # prediction -> decision
```

**context** is spread as kwargs into the oracle, **extra** never touches the solver.

# Algorithms in DFL Algorithms

The right algorithm depends on the considered **problem**:

|                   | Imitation                      | ERM                                  |
|-------------------|--------------------------------|--------------------------------------|
|                   | fit precomputed targets        | minimize expected cost               |
| <b>static</b>     | Fenchel-Young imitation        | Structured RL                        |
| <b>stochastic</b> | <b>Fenchel-Young imitation</b> | <b>MirrorDescent</b> , Structured RL |
| <b>dynamic</b>    | Anticipative imitation, DAgger | Structured RL                        |

Imitation corresponds to the supervised setting seen earlier (a target solution is available), ERM to expected-cost minimization (only the cost is available). The static, stochastic, and dynamic rows refer to whether the problem instance has scenarios or a multi-stage structure.

- **Today** is **stochastic**: imitation, then ERM.
- **Structured RL**: coming soon.

## Training with an algorithm

Bundle the predictor and oracle into a `DFLPolicy`, pick an algorithm, train.

```
# 1. policy = predictor + CO oracle  
policy = DFLPolicy(model, maximizer)
```

```
# 2. a decision-focused training algorithm  
algo = PerturbedFenchelYoungLossImitation(; nb_samples=10,  $\varepsilon=1.0$ )
```

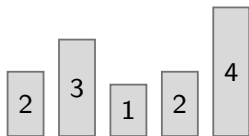
```
# 3. train end to end (metrics logged each epoch)  
history = train_policy!(  
    algo, policy, train_data; epochs=50, metrics=(gap_metric,)  
)
```

Same call, **swap the algorithm**: `MirrorDescent`, `Dagger`, ...

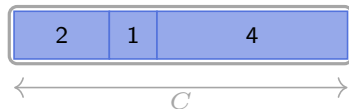
- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 DFL for stochastic tail assignment
- 4 The JuliaDecisionFocusedLearning ecosystem
- 5 Demo: contextual stochastic knapsack problem**
- 6 Practice session 2

# The contextual stochastic knapsack problem

- **Instance:**  $n$  items with integer weights  $c$  and a capacity  $C$ .
- **Context**  $x$ : observed features correlated with the item values.
- **Scenario**  $\xi \in \mathbb{R}^n$ : the **realized item values**, drawn at random and revealed *after* we decide:  $\xi = u_{\text{base}} + M x_{\text{raw}} + \text{noise}$ , with  $M$  unknown.
- **Decision**  $y \in \{0, 1\}^n$ : which items to take, under  $c^\top y \leq C$ .

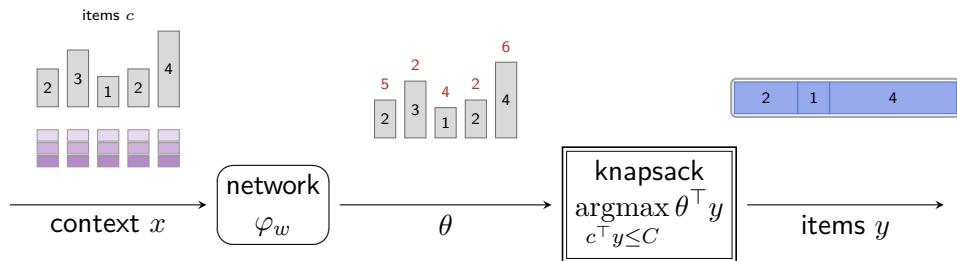


Instance: items with weights  $c$



Decision  $y$ : chosen items,  $c^\top y \leq C$

# The DFL policy



- **Policy**  $\pi_w(x) = f(\varphi_w(x))$ .
- **Target**  $\bar{y}$ : the *anticipative* solution (the knapsack solved on the realized values  $\xi$ ).
- **Learn** the weights  $w$  by Fenchel-Young imitation.

## Live demo

→ 04\_dfl\_demo\_knapsack.jl

## Practice session 2

→ 05\_dfl\_notebook.jl